

kci-dev:

What Changed, What Works, and What Kernel Developers Still Need

Arisu Tachibana

kci-dev creator and project lead

Senior Engineer, Cybertrust Japan Co., Ltd.

Kernel Testing & Dependability MC / LPC 2026¹

Our work since LPC 2025

- **Reliable automation:** fixed hangs in job watching¹ and bisection.²
- **Clearer results:** corrected false success reports³, validation⁴ and JSON output.⁵
- **Python API:** lets existing tools use kci-dev without parsing terminal output.⁶
- **Broader workflows:** added external build reporting⁷ and patch submission.⁸

A maintainer's question

What changed for my kernel?

- Which revision and configuration did we test?
- Which failures appeared with my change?
- Do we have enough evidence to act?

kci-dev brings KernelCI data into scripts and review workflows.^{1,2}

What changed since LPC 2025

Available by v0.1.11	What it enables
Python client	Import kci-dev into other tools ¹
KCIDB submission + storage	Report external builds and upload artifacts ²
Patchset submission	Test local patches or allowed patch URLs ³
Validation + workflow fixes	Check consistency and improve automation ¹

Current main: structured comparison reports and CI gates⁴

Source snapshot: 6 October 2026, `e4c0087`. MCP remains experimental.⁵

Would you trust this result?

Recorded two-patch smoke test, staging.¹

Signal	Captured value
Watcher warning	failed with status: FAIL ¹
Stored build result	state=available , result=pass ¹
Watcher exit status	0 ¹

Which signal should an automated decision use?

Why those answers disagreed

Client path	Interpretation in the recorded revision
Non-root state classifier	<code>available/pass</code> becomes <code>FAIL</code> ¹
Selected-test exit path	<code>result=pass</code> returns exit <code>0</code> ²

Replaying the classifier reproduces the warning's decision.³

State, outcome and exit status need a consistent contract.

Patched-source verification remained incomplete in this smoke test.³

A reusable Python interface

```
from kcidb import KernelCIClient

client = KernelCIClient()
summary = client.get_summary(
    origin="maestro", giturl=GIT_URL,
    branch=BRANCH, commit=COMMIT,
)
```

Dashboard queries return Python objects.¹

`submit_build(...)` reports an external build to KCIDB.²

Example plugins using the Python API

Application	Workflow it adds	Shared client methods
<code>kci-patchwork</code>	Test a Patchwork series on a known base	<code>trigger_patchset</code> , <code>get_node</code> , <code>get_nodes</code> ^{1,2}
<code>kci_release_review</code>	Compare tested revisions and publish reports	<code>compare_results</code> , <code>get_build</code> , <code>get_test</code> , <code>get_log</code> ^{3,4}

Standalone Python applications importing `KernelCIClient` .^{1,3}

Patches on a known base

```
kci-dev patchset --nodeid "$CHECKOUT_NODE" \  
  --patch 0001-fix.patch \  
  --patch 0002-test.patch \  
  --job-filter "$JOB_FILTER" \  
  --watch --test "$TEST_PATH"
```

Requires an existing checkout and a configured pipeline/token.^{1,2}

`--patchurl` accepts allowed URLs, including Patchwork mbox URLs.¹

Choosing the base and the jobs

- The checkout identifies the unpatched source revision and tarball.¹
- The series must apply to that revision.¹
- Job filters select builds and tests available on that instance.¹
- The configuration must exercise the code the series changes.²

A passing build provides evidence for the configuration that ran.²

kci-patchwork: a series workflow

1. Resolve the Patchwork URL to a complete series.¹
 2. Validate patch order and freeze the exact diffs.²
 3. Record the base, selected jobs and submission identifiers.³
 4. Collect the patched tree and compare completed results.⁴
- `manifest.json` records the inputs.³ HTML and JSON reports retain the evidence.⁵

CIP series submission

```
kci-patchwork \  
  "https://patchwork.kernel.org/project/cip-dev/list/?series=1178390" \  
  --config ~/.config/kci-dev/kci-dev.toml \  
  --instance staging \  
  --checkout 6ab272040ed308c6c2d206c3 \  
  --job kbuild-gcc-14-arm64-510-cip \  
  --submit
```

Captured command: one series, an explicit base and a selected CIP build.^{1,2,3}

First capture: the series was running

```
{
  "status": "RUNNING", "exit_code": 3,
  "terminal": false,
  "counts": {"total": 2, "descendants": 1,
    "states": {"closing": 1, "running": 1},
    "results": {"pass": 1, "unset": 1}},
  "comparison": {"performed": false, "complete": false}
}
```

Submission and result collection are active.^{1,2}

The completed build comparison is still pending in this capture.^{1,3}

Continuing the same Patchwork run

```
RUN=runs/patchwork.kernel.org/series-1178390  
kci-patchwork status --run "$RUN"  
kci-patchwork watch --run "$RUN"  
xdg-open "$RUN/report.html"
```

- `report.html`: results and comparisons; captured snapshot^{1,2}
- `report.json`: nodes, classifications and collection errors²
- The saved run retains the submitted node and tree IDs.³

CIP series 1178390: the build completed

Observed stage	Final state	Result
Apply the frozen patch	done	PASS ¹
Build the ARM64 kernel	done	PASS ¹
Build kselftest binaries	done	PASS ¹

Patchwork diff and pipeline artifact: **573 identical bytes.**¹

Read back on 6 October: `terminal=true`, three passing nodes.¹

Three PASS nodes. Is the review complete?

Baseline comparison	Observed count
Matched PASS to PASS	2 ¹
Missing in the patched run	3,699 ¹
Ambiguous / incomplete comparisons	9 / 15 ¹

Report: **EVIDENCE_INCOMPLETE**, exit 2.¹

The selected builds passed. The baseline also contains runtime tests.¹

Choose required coverage before using the report to approve a series.

Patchset identity in the Python API

```
response = client.trigger_patchset(  
    nodeid=base_id, patches=ordered_diffs,  
    job_filter=[job],  
)  
patchset = response["node"]  
root = client.get_node(patchset["id"])
```

Patched and unpatched trees can share a Git commit hash.^{1,2}
Collection also uses the patchset hash, tree ID and parent chain.^{3,4}

Comparing two revisions

```
kci-dev results compare \  
  --giturl "$GIT_URL" --branch "$BRANCH" \  
  --format json "$BASE" "$HEAD"
```

Current main: explicit full commit hashes and structured output^{1,2}

Matches origin, platform, arch, compiler, config and path.³

Keeps repeated results and their IDs.³

What the report tells us

Category	Interpretation
<code>regression</code>	PASS becomes FAIL/ERROR, or history signal ¹
<code>fixed</code>	Recovery to PASS ¹
<code>persistent_fail</code>	Failure on both revisions ¹
<code>unstable</code>	History signal or another status change ^{1,2}
<code>new</code>	Result appears only in HEAD ¹
<code>missing</code>	Result appears only in BASE ¹

A regression label needs investigation.

`FAIL` and `ERROR` both count as failures in this classifier.¹

kci_release_review: tested revisions

- Select a baseline and candidate by full commit hash.¹
- Reuse kci-dev's comparison and result classifications.¹
- Collect selected result details and bounded log excerpts.¹
- Publish portable HTML with the complete JSON report.²

The application queries existing results through the Python API.¹

A report built on the shared client

```
from kci_release_review.client import ReviewClient
from kci_release_review.report import collect_report

client = ReviewClient(max_issue_lookups=20)
report = collect_report(
    client, selection, max_evidence=3, log_bytes=8192,
)
```

`ReviewClient` extends `KernelCIClient`.¹

`collect_report()` calls `compare_results()` and keeps its evidence.²

Published release-review results

linux-6.6.y: v6.6.157 to v6.6.158^{1,2}

Regression 1	Fixed 1	Unstable 0
Persistent Fail 870	New 2573	Missing 1047

Status: **REVIEW_REQUIRED**. Report collected on 5 October 2026.^{3,4}

[Open the published reports](#)

One candidate to investigate

kernelci_watchdog_reset.wdt-reset.wdt-get-timeout¹

Context	Captured result
Platform	mt8195-cherry-tomato-r2 ¹
Architecture / compiler	ARM64 / GCC 14 ¹
Baseline	PASS ¹
Candidate	FAIL ¹
Known issues	Not checked in this report ²

The report links both result IDs for follow-up.³

A CIP result with no review signals

v6.12.108-cip31 to v6.12.111-cip32¹

Observed results	Baseline	Candidate
Builds	6 PASS	6 PASS ²
Boots	0	15 PASS ²
Tests	0	89 PASS, 11 SKIP ²

115 new entries, with no observed failures or missing results.²
Required coverage remains unassessed.³

Incomplete evidence stays visible

linux-6.12.y: v6.12.110 to v6.12.111

- The report retains 1 regression candidate and 36 recoveries.¹
- Returned tree history belongs to a different candidate.¹
- The issue budget leaves 370 result IDs unchecked.¹

Status: **EVIDENCE_INCOMPLETE**, exit code **2**.^{2,3}

The exact-commit results remain available for review.⁴

Gate policy and coverage

```
kci-dev results gate \  
  --giturl "$GIT_URL" --branch "$BRANCH" \  
  --base "$BASE" --head "$HEAD" \  
  --fail-on regression,missing --format json
```

Exit	Meaning in current main
0	No selected category triggered the policy ¹
1	A selected category triggered the policy ¹
2	Incomplete comparison or command error ¹

Expected coverage still needs an explicit test plan.²

What the plugins expose

- Release review checks that history belongs to the selected head.¹
- Patch review keeps patchset and baseline node identities separate.^{2,3}
- Reports preserve partial results, skipped work and collection errors.^{1,2}
- Shared API fixes can benefit every application using the client.⁴

Consistent configuration and result identity remain core requirements.^{5,4}

A contract for the next plugin

Shared kci-dev interface	Application responsibility
Service requests and typed failures ¹	Input selection and recovery
Explicit endpoint configuration ¹	Credential and instance choices
Result identities and classifications ¹	Required coverage and review policy
Structured Python data ¹	Reports and integration with other tools ^{2,3}

Pin the library revision^{4,5} and retain the inputs behind each report.^{2,3}

MCP as an experimental interface

```
pip install 'kci-dev[mcp]'  
kci-dev mcp
```

- Query results, hardware and known issues¹
- Inspect Maestro nodes with configured API access²
- Trigger or retry jobs with pipeline access and a token²

Tool names and response formats can change.³

Proposed priorities

Workstream	Concrete next contribution
Shared API correctness	Scope history and propagate configuration ¹
Patch review	Validate completed comparisons on real series ²
Required coverage	Define jobs, platforms and completion rules ³
Review integration	Connect reports to maintainer workflows ³

Both example plugins provide starting points for this work.^{2,3}

Decisions for this MC

- Which real series or release should we validate next?¹
- Which jobs and platforms are required for that workflow?
- Which guarantees should become part of the shared Python API?
- Who can review the resulting evidence with us?

kci-dev² / kci-patchwork³ / kci_release_review⁴